Lecture 3

Class Design & Encapsulation In Object-Oriented Programming

Previously on PR2…

- In the previous lecture, you learned about:
  - Why use OOP?
  - Object, attributes and behavior
  - Class
  - Instances in Java
  - Class as a 3-compartment box
  - The dot (`.`) operator
  - Constructors
  - Primitives vs References
  - Static vs non-static

Instance Variables

- An instance variable has a name (or identifier) and a type; and holds a value of that particular
  - **Naming Convention:** A variable name shall be a *noun* or *noun phrase*. The first word is in lowercase and the rest of the words are initial-capitalized (camel-case).

```
[AccessControlModifier] type variableName [= initialValue];
```

```
[AccessControlModifier] type variableName-1 [= initialValue-1]
[, type variableName-2 [= initialValue-2]] ... ;
```

E.g.

```
private double radius;
public int length = 2, width = 1;
```

Instance Methods

- A method:
  - receives arguments from the caller,
  - performs the operations defined in the method body, and
  - returns a piece of result (or void) to the caller.
- The syntax for method declaration in Java is as follows:

```
[AccessControlModifier] returnType methodName ([parameterList]) {
   // method body or implementation
   ......
}
```

Instance Methods

- Method Naming Convention:
  - A method name shall be a verb, or a verb phrase made up of several words. The first word is in lowercase and the rest of the words are initial-capitalized (camel-case).
  - For example: `getArea()`, `setRadius()`…

```
// return the area of this Circle instance
public double getArea() {
   return radius * radius * Math.PI;
}
// return type is void => does not contain a return value
public void setWidth(double width) {
      this.width = width;
}
```

OOP Example: `Circle` class

Class Definition

```
Circle
-radius:double=1.0
-color:String="red"
+Circle()
+Circle(r:double)
+Circle(r:double,c:String)
+getRadius():double
+getColor():String
+getArea():double
```

Instances

```
c1:Circle
-radius=2.0
-color="blue"
+getRadius()
+getColor()
+getArea()
```

```
c2:Circle
-radius=2.0
-color="red"
+getRadius()
+getColor()
+getArea()
```

```
c3:Circle
-radius=1.0
-color="red"
+getRadius()
+getColor()
+getArea()
```

Source code: `Circle.java`

```java
public class Circle {
   // instance variables
   private double radius;
   private String color;
   public Circle() { // default constructor
      radius = 1.0;
      color = "red";
   }
   public Circle(double r, String c) { // another constructor
      radius = r;
      color = c;
   }
   public double getRadius() { return radius; }
   public String getColor() { return color; }
   public double getArea() {
      return radius * radius * Math.PI;
   }
}
```

Example test source code: `TestCircle.java`

```java
public class TestCircle {
   public static void main(String[] args) {
      // Create Circle instances
      // Use default constructor
      Circle c1 = new Circle();
      // Use the other constructor
      Circle c2 = new Circle(2.0, "blue");
      // invoke instance methods
      System.out.println("The radius is: " + c2.getRadius());
      System.out.println("The color is: " + c2.getColor());
      System.out.printf("The area is: %.2f%n", c2.getArea());
   }
}
```

Method Overloading (Revisit)

- Method overloading allows a method name to have multiple implementations (version).
- These implementations must be distinguishable by their parameter list.
- Parameters can differ in number, type, or order.

Method Overloading

```
public class MyClass {
    void method() {}
    void method(int x) {}
    void method(float x) {}
    void method(int x, double y) {}
}
```

- In class `MyClass`, there are 4 methods with the same name but different parameters

Method Overloading

- **Example**: The method `average()` has 3 versions, with different parameter lists. The caller can invoke the chosen version by supplying the matching arguments.

```
public class MethodOverloadingTest {
   public static int average(int n1, int n2) { // version 1
      System.out.println("Run version 1");
      return (n1 + n2) / 2;
   }
   public static double average(double n1, double n2) {
      // version 2
      System.out.println("Run version 2");
      return (n1 + n2) / 2;
   }
   public static int average(int n1, int n2, int n3) { // version 3
      System.out.println("Run version 3");
      return (n1 + n2 + n3) / 3;
   }
}
```

Method Overloading

```
public static void main(String[] args) {
   System.out.println(average(1, 2));
   // run version 1
   // 1
   System.out.println(average(1.0, 2.0));
   // run version 2
   // 1.5
   System.out.println(average(1, 2, 3));
   // run version 3
   // 2
   System.out.println(average(1.0, 2));
   // run version 2 (int 2 implicitly casted to double 2.0)
   // 1.5
   // average(1, 2, 3, 4);
   // compilation error: no suitable method found
   // for average(int,int,int,int)
}
```

Constructors (Revisit)

- A constructor is a special method used to create objects
- Characteristics of a constructor:
  - It has the same name as the class
  - It has no return type in its method heading. It implicitly returns `void`
  - It does not return a value
  - It can only be invoked via the `new` operator
  - Constructors are not inherited (explained later)

Example of constructor

```java
public class Circle {
    private double radius;
    private String color;
    public Circle() {
        radius = 1.0;
        color = "red";
    }
    public Circle(double r) {
        radius = r;
        color = "red";
    }
    public Circle(double r, String c) {
        radius = r;
        color = c;
    }
}
```

Class

```java
Circle c1 = new Circle();
// use 1st constructor
Circle c2 = new Circle(2.0);
// use 2nd constructor
Circle c3 = new Circle(3.0, "red");
// use 3rd constructor
```

Instances

Contructor Overloading

- Constructor, like an ordinary method, can also be overloaded.
  - The `Circle` class has three overloaded versions of constructors differentiated by their parameter list, as followed:

```
Circle() // the default constructor
Circle(double r)
Circle(double r, String c)
```

Overloading

- Depending on the actual argument list used when invoking the method, the matching constructor will be invoked.
- If your argument list does not match any one of the methods, you will get a compilation error.
  - **Note:** C language does not support method overloading. You need to use different method names for each of the variations. C++, Java, C# support method overloading.

`this` keyword

- `this` is used to refer to this instance inside a class definition.
- `this` is used within a class to reference the members of the class **(fields and methods)**
- Use `this.field` to differentiate between the class' field and local variables or method parameters

`this` keyword

```
public class Circle {
   double radius;
   public Circle(double radius)
      this.radius = radius;
      // "radius = radius" does not make sense!
      // "this.radius" refers to an instance's member
      // "radius" resolved to the method's parameter
   }
   ...
}
```

- Java offers a keyword called `this` to address such naming conflicts.
- Using `this.radius` specifies the member variable, whereas simply `radius` refers to the method's argument.

Package

- Package is used to divide classes and interfaces into different groups.
- This job is similar to putting files in folders on the hard disk
- The following example creates class `MyClass` in package `lect3`;

```
package lect3;

public class MyClass {

}
```

Package

- Similar to hard disk files & folders structure, we can create sub-packages inside packages
- In Java there are many packages divided by function
  - `java.util`: contains utility classes
  - `java.io`: contains data input/output classes
  - `java.lang`: contains commonly used classes…

Importing package

- The import statement is used to indicate that the class has been defined in a package
- Classes in the `java.lang` package and classes defined in the same package as the using class will be imported by default

```
package lect.lect4;
import lect.lect3.MyClass;
import java.util.Scanner;
public class HelloWorld {
      public static void main(String[] args) {
      MyClass obj = new MyClass();
      Scanner sc = new Scanner(System.in);
… }}
```

The Access Control Modifiers

- An access control modifier can be used to control the visibility of a class, or a member variable or a member method within a class
- In java there are 4 different characteristics:
  - `private`: allowed to be used only internally within the class
  - `public`: completely public
  - {default}:
    - It is public for classes accessing the same package
    - Is private from other export packages of the classes
  - `protected`: similar to {default} but allowed to design even if the subclass and package are different

The Access Control Modifiers

- The level increases gradually in the direction of the arrow

public → protected → {default} → private

Access Levels

Modifier	Class	Package	Subclass	World
`public`	Y	Y	Y	Y
`protected`	Y	Y	Y	N
no modifier	Y	Y	N	N
`private`	Y	N	N	N

The Access Control Modifiers

```
package p1;
public class A{
    public int a;
    protected int b;
    int c;
    private int d;
}
```

use

use

extends

```
package p1;
public class B{
    A x = new A();
    void method(){
        x.a = 1;
        x.b = 1;
        x.c = 1;
        x.d = 1;
    }
}
```

```
package p2;
public class C{
    A x = new A();
    void method(){
        x.a = 1;
        x.b = 1;
        x.c = 1;
        x.d = 1;
    }
}
```

```
package p3;
public class D extends A{
    void method(){
        a = 1;
        b = 1;
        c = 1;
        d = 1;
    }
}
```

Non-Encapsulation

- Suppose you define the `Student` class and declare name and score as follows
  - When using, users can assign data to fields arbitrarily
  - What if valid scores are only from 0 to 10

```
public class Student{
    public String name;
    public double score;
}
```

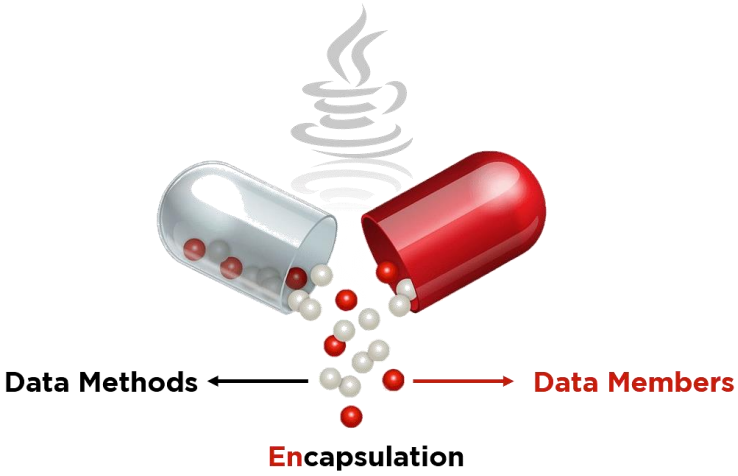
```
public class MyClass{
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Emma Watson";
        s.score = 20.5;
}}
```

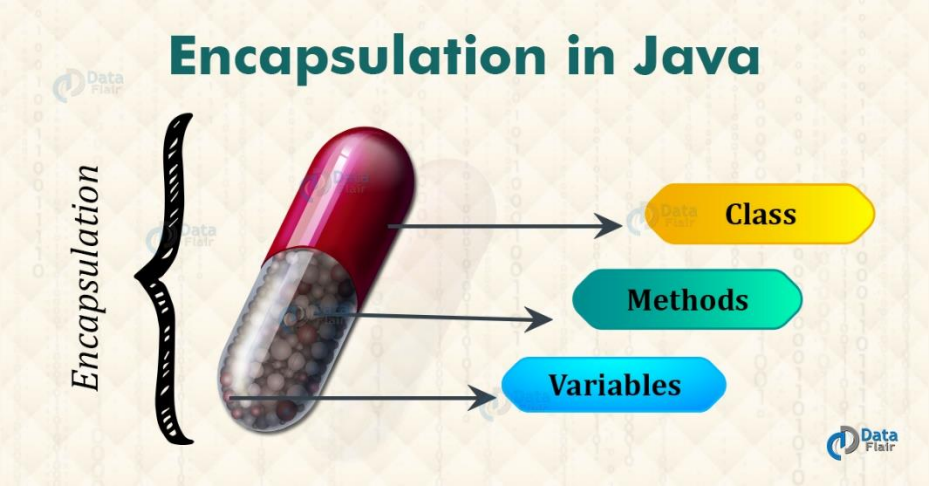
Information Hiding

- **Definition:** A design principle that restricts access to an object's internal details.
- **Purposes:**
  - Protect object integrity
  - Reduce system complexity
  - Prevent unwanted changes
  - Allow internal changes without affecting external codes
- How to achieve information hiding in Java?
  - ➢ Use *access modifiers* to control visibility

Encapsulation

- Encapsulation combines data and related operations into a single unit and restricts access to the object's internal state.
  - Hidden data fields (use methods to retrieve data)
  - Ensures data integrity
  - Allows validation before modifying data
  - Supports modular, reusable design





Encapsulation

- To hide information, use `private` for data fields

```
private double score;
```

- Add getter and setter methods to read and write private fields

```
public void setScore(double score) {
    this.score = score;
}
public String getScore(){
    return this.score;
}
```

Encapsulation

```java
public class Student {
    private String name;
    private double score;
    public void setName(String name) {
        this.name = name;
    }
    public String getName() {
        return this.name;
    }
    public void setScore(double score) {
        if (score < 0 || score > 10) {
            System.out.println("Score invalid");
        } else {
            this.score = score;
        }
    }
    public double getScore() {
        return this.score;
    }
}
```

Just add code to the `setScore()` method to handle invalid data

Encapsulation

```
public class MyClass {
   public static void main(String[] args) {
      Student sv = new Student();
      sv.setName("Emma Watson");
      sv.setScore(8.5);
   }
}
```

Method `toString()`

- A class should contain a public `toString()` method
- toString() method: Provides a string description of the class instance
- Invocation: `anInstanceName.toString()`
- Implicit invocation: Through `println()` or the `+` operator
- `println(anInstance):` Implicitly triggers the `toString()` method for that instance

Method toString()

```
// return a String description of this instance
public String toString() {
   return "Name: " + name + ", score: " + score;
}
```

```
public class MyClass{
   public static void main(String[] args){
      Student sv = new Student();
      sv.setName("Emma Watson");
      sv.setScore(8.5);
      System.out.println(sv.toString());
      // Name: Emma Watson, score: 8.5
   }
}
```

Constants in Java

- A constant is a variable declared with the `final` modifier.
  - can be assigned only once
  - must be initialized when declared
  - its value cannot be modified after initialization
- Example:

```java
public class JavaConstants {
    public final double X_REFERENCE = 1.234;
    private final int MAX_ID = 9999;

    public void someMethod() {
        this.MAX_ID = 10000;
        // Error: cannot assign value to final variable MAX_ID
    }
}
```

Notes on `final`

- Naming convention:
  - Constant names are nouns or noun phrases.
  - Uppercase words separated by underscores.

    Examples: `X_REFERENCE, MAX_INTEGER…`
- Some behaviors:
  - A final primitive variable cannot be reassigned a new value.
  - A final reference variable cannot take a new object, but the object's internal state may still change.

Lecture summary

- Instance Variables
- Instance Methods
- Method Overloading
- The Access Control Modifiers
- Encapsulation
- Method `toString()`
- Java Constants